ChatGPT

Instruction for Building a Production-like Educational Cryptocurrency Blockchain Network

Overview

This document describes the requirements, architecture, technology stack, communication design, logging/audit plan and UI/UX guidelines for building a comprehensive educational cryptocurrency blockchain network. The goal is to mimic production-grade networks such as Bitcoin or Ethereum while remaining accessible for educational purposes. The network should support user registration with KYC/AML compliance, secure transaction processing, node-to-node communication via a containerised microservices architecture, and a polished user interface similar to mainstream crypto wallets.

Features

Core blockchain

- **Distributed ledger and consensus** – Each node maintains a copy of the blockchain. Blocks include a hash linking to the previous block to ensure immutability [1]. Choose a consensus algorithm (Proof-of-Authority or a simplified Proof-of-Stake) suitable for a small network. Support block creation, propagation and chain reorganisation.

- **P2P network layer** – Nodes discover peers, broadcast transactions and blocks, and handle network partitions. Use asynchronous gRPC or WebSockets for network communication.

- **Transaction lifecycle** – Implement wallets for creating and signing transactions, a mempool to queue unconfirmed transactions, and block validation to ensure correct signatures and sufficient balances before inclusion.

- **Smart contract layer** – Optional: allow user-defined scripts or smart contracts (for example, simple token transfer contracts). This helps illustrate how programmable blockchains differ from basic payment chains.

- **Cryptographic wallet management** – Provide hot, warm and cold wallet types [2] with secure key storage. Use Python's `cryptography` library for key generation, ECDSA signatures and multi-signature support. Expose APIs to generate new addresses, export public keys and sign transactions.

- **KYC/AML compliance** – Integrate a KYC module that stores customer identity proofs off-chain and records hashes on the blockchain for integrity. Blockchain-based KYC offers decentralised data storage, enhanced security, immutable records and improved efficiency [3] [4]. The authentication

service should handle user registration, multi-factor authentication (MFA), role-based access control and interact with external KYC vendors [5].

- **Account management and authentication** – Users can register, log in, manage profiles and view wallets. Use secure password hashing (bcrypt or Argon2) and OAuth. Provide identity verification flow (upload ID, selfie) for KYC [6].

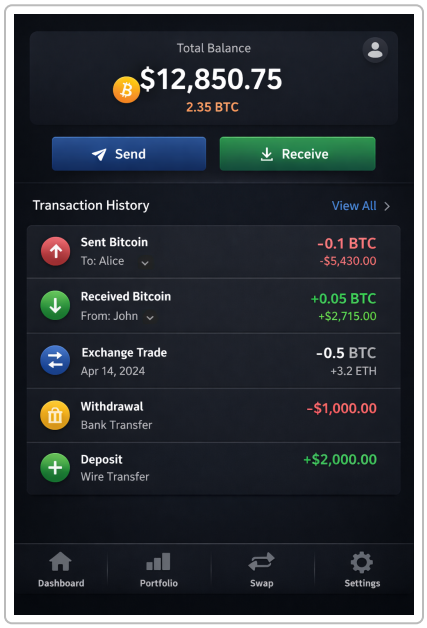
- **Trading/exchange features (optional)** – To emulate an exchange-like experience, include modules such as:

- **Order management system (OMS)** for market, limit and stop orders [7].
- **Matching engine** for low-latency trade matching with sub-millisecond execution [8].
- **Trade settlement layer** to update balances, calculate fees and record transactions [9].
- **Liquidity management service** to provide market-making bots and integrate external liquidity providers [10].
- **API endpoints** for retrieving order books, placing orders and querying trade history.

Front-end (UX/UI)

- **Responsive web/mobile interface** – Provide a web application (React or Next.js) and optionally mobile apps (Flutter). Display real-time balances, transaction history, order books and market charts using WebSockets [11]. The UI should be intuitive and visually appealing.

- **User dashboard** – Show wallet balance, open orders, KYC status and quick actions like **Send** and **Receive**. Provide deposit/withdrawal options and a portfolio overview [12]. An example design is shown below:



- **Onboarding with education** – Analyse end-user needs and competitors; provide clear instructions, in-app tutorials and educational resources to make crypto concepts accessible to beginners [13].

Visualise data with charts and graphs to help users understand portfolio balances and market trends [14].

- **Trust and security design** – Communicate security measures and enforce data encryption, secure session management and automated logout to protect sessions [15]. Support biometric authentication and device fingerprinting [16].

- **Accessibility and customisation** – Support light/dark themes, multi-language localisation and accessible design (WCAG 2.1 compliance). Ensure the UI scales across desktop, tablet and mobile displays.

Administrative/Back-end features

- **Real-time analytics and monitoring** – Provide dashboards for administrators to view user activity, network statistics and node health [17]. Integrate monitoring tools like Prometheus and Grafana for metrics [18].

- **Wallet management and liquidity control** – Administrators can manage user wallets and configure fee structures [19].

- **Security tools** – Integrate IP whitelisting, AML alerts and fraud detection [20]. Add DDoS protection, a web application firewall (WAF), anti-phishing measures, secure API keys and multi-signature authorisation [21].

- **Multi-chain support** – Design the architecture to integrate additional blockchains (Bitcoin, Ethereum) through dedicated full nodes and transaction parsers [22].

- **Governance features** – Implement role-based permissions, multi-signature consensus for administrative operations and upgradeable protocols. Provide optional on-chain voting for protocol changes.

- **Notification system** – Send price alerts, transaction confirmations and system messages via email/SMS/push notifications [23].

Technology Stack

The project should adopt a **microservices architecture** to ensure flexibility, scalability and fault isolation. The following stack is recommended:

Layer/Service	Recommended Technology	Rationale
API Gateway	Kong, Traefik or NGINX [24]	Provides a single entry point, rate limiting, authentication and routing.

Layer/Service	Recommended Technology	Rationale
Authentication & User Service	Python FastAPI/Django, JWT/OAuth, Celery, PostgreSQL	Handles registration, login and KYC workflows [5].
Wallet Service	Python (FastAPI + cryptography), gRPC, HSM integration	Manages hot/warm/cold wallets [2], key generation and signing.
Transaction/ Blockchain Node Service	Python (asyncio), custom P2P protocol (gRPC/WebSocket), consensus algorithm	Maintains blockchain state, mempool and consensus; synchronises with peers.
Matching Engine & OMS	Performance-critical languages (Rust/Go) or Python with C extensions	Executes order matching with sub-millisecond latency [8].
Settlement Service	Python + SQLAlchemy + ACID database	Updates balances and records transaction history [9].
Liquidity Service	Python + market-making bots + integration to external exchanges [10]	Provides liquidity and cross-exchange order mirroring.
KYC Module	Python microservice integrating with external KYC providers	Stores only hashes of identity documents on chain; off-chain storage for actual documents [3].
Notification Service	Python + Celery + Redis	Sends emails/SMS/push notifications.
Database Layer	PostgreSQL/MySQL for user data and balances [25]; Redis for caches [26]; MongoDB for logs	Provides ACID transactions and high-performance caches.
Messaging	Kafka or RabbitMQ	Enables asynchronous communication between services (transaction propagation, order events).
Logging & Monitoring	Docker logging drivers, Fluentd/ Logstash → Elasticsearch, Kibana or Grafana [27] [28]; Prometheus + Grafana for metrics [29]	Centralises logs and metrics; supports search, visualisation and alerting.
Front-end	React/Next.js or Flutter	Implements responsive UI; uses WebSockets for real-time updates.
Containerisation	Docker and Docker Compose or Kubernetes	Each microservice runs in its own container; orchestrated for scalability and isolation.

Python is chosen for most microservices due to its readability, rich ecosystem (FastAPI, cryptography, SQLAlchemy) and ease of prototyping. However, latency-critical components (matching engine, consensus) should be implemented in performance-oriented languages such as Rust or Go [30].

Inter-Node and Service Communication

Docker Networking

Microservices require isolation and controlled communication. **Avoid placing all services on a single network**; a flat network gives every container access to every other container and violates the principle of least privilege [31]. Instead, design **tiered networks**:

1. **Edge (DMZ) Tier** – Contains public-facing reverse proxy/API gateway. Only this tier is exposed to the Internet [32].
2. **Application Tier** – Hosts front-end services and the API gateway, which routes requests to backend services [32].
3. **Service Tier** – Contains core microservices (user, wallet, order, settlement, KYC) that communicate via internal networks. These services cannot be reached directly from outside the gateway [33].
4. **Data Tier** – Databases (PostgreSQL, Redis) reside in isolated networks; each service has exclusive access to its own database to enforce **database-per-service isolation** [34].

Use Docker Compose to define separate networks and ensure that a service can reach only its allowed peers. For cross-project communication (e.g., connecting nodes across multiple compose files), create external networks [35].

P2P Communication

Blockchain nodes communicate using a custom P2P protocol over TCP/WebSocket. Nodes discover peers using a bootstrap node list or gossip protocol. Use gRPC for remote procedure calls (block requests, transaction propagation) and publish/subscribe patterns for new block/transaction announcements.

Service-to-Service Communication

For synchronous calls (e.g., user service verifying KYC), use REST or gRPC with TLS. For asynchronous events (e.g., new transaction created or order executed), publish messages to Kafka/RabbitMQ topics. Apply circuit breakers, retries and timeouts to improve reliability.

Logging and Audit Strategy

Logging is crucial in microservices because each interaction constitutes its own event [36]. **Treat logs as event streams**: each process writes unbuffered logs to `stdout`, and the execution environment captures them [37]. In containers, file systems are destroyed on restart, so treat logs as streams and ship them to a central system [28].

Implementation

1. **Configure log drivers** – Use Docker logging drivers (json, syslog or GELF) to forward logs from each service to Fluentd or Logstash [38].
2. **Centralise logs** – Use Logstash to collect, filter and parse logs; store them in Elasticsearch [39]. Visualise logs in Kibana or Grafana. Provide dashboards for error rates, transaction counts, block times and KYC events.
3. **Audit logging** – Record critical actions (account creation, KYC approval, transactions, admin operations). Audit logs must contain event name, timestamp, actor/service ID, impacted entity and source IP [40]. Audit trails are immutable and used for compliance and forensic analysis [41].
4. **Metrics and alerts** – Use Prometheus to collect metrics (CPU, memory, response times, block times) and Grafana to visualise them. Configure alerting rules for anomalies like high error rates or slow block confirmation.

Implementation Instructions

1. **Repository structure** – Create a mono-repo with directories for each microservice (`api-gateway/`, `auth-service/`, `wallet-service/`, `blockchain-node/`, `order-matching/`, `settlement/`, `liquidity/`, `kyc-service/`, `notification/`, `frontend/`, `logging/`).
2. **Define Docker Compose** – Write a `docker-compose.yml` with separate networks (`edge`, `services`, `user-data`, `order-data`, etc.) following the tiered pattern [42]. Configure environment variables and volumes for persistent storage.
3. **Implement Authentication Service** – Use FastAPI with endpoints for registration, login and KYC submission. Integrate with an external KYC provider (e.g., AuthBridge). Hash passwords with bcrypt and store KYC metadata in PostgreSQL.
4. **Implement Wallet Service** – Provide API methods to generate addresses, sign transactions, manage balances and query blockchain state. Use Python's `cryptography` library and integrate a hardware security module (HSM) for key storage. Support different wallet types (hot, warm, cold) [2].
5. **Implement Blockchain Node** – Use asyncio to build a peer-to-peer layer and consensus algorithm. Maintain a mempool, create blocks at regular intervals, validate transactions and propagate blocks to peers. Persist blockchain state in a local database (e.g., LevelDB or PostgreSQL). Record hashed KYC proofs on chain.
6. **Order Management and Matching Engine (optional)** – Build an OMS microservice to queue and validate orders and a matching engine (preferably in Rust or Go) for high performance [8].
7. **Settlement Service** – Handle updating user balances and recording completed trades. Ensure ACID compliance [9]. Provide a ledger view for admins.
8. **KYC Service** – Accept user identity documents, store them securely off-chain (encrypted object storage), compute a hash and record the hash on the blockchain. Use smart contracts to store hashed KYC data and grant authorised parties access.
9. **Front-end** – Use React or Next.js to build a responsive UI similar to mainstream wallets. Connect to the API gateway via REST/WebSockets for real-time updates. Implement KYC forms, dashboards, transaction views and educational content as described in the UI/UX guidelines.
10. **Logging and Monitoring** – Deploy the ELK stack via Docker Compose; configure microservices to forward logs; build Kibana dashboards. Use Prometheus and Grafana for metrics and set up alerts for high error rates or latency.

11. **Testing and Deployment** – Write unit and integration tests. Use Docker Compose for local development; scale to Kubernetes for production-like deployment. Employ CI/CD to automate builds, tests and deployments.
12. **Documentation and API specification** – Provide OpenAPI specifications for each microservice. Document architecture, network topology and deployment instructions. Include diagrams illustrating the tiered network and microservices interactions.

Security and Compliance Considerations

- **Transport security** – Enforce TLS for all communication channels (REST, gRPC, WebSocket).
- **Authentication security** – Use JWT tokens with short expiry and refresh tokens; implement MFA and biometric authentication [5].
- **Network security** – Apply IP whitelisting, rate limiting and intrusion detection in the API gateway [43]. Use network policies to restrict lateral movement.
- **Application security** – Use OWASP best practices to defend against injection, XSS and CSRF. Perform code reviews and vulnerability scanning.
- **Data security** – Ensure that no sensitive KYC data is stored directly on the blockchain; only hashed pointers are recorded [3]. Encrypt data at rest and in transit.
- **Operational security** – Conduct regular security audits and penetration testing. Monitor logs for suspicious activity and maintain an incident response plan.

Differences between this Educational Project and Production Networks

The `diff.md` file provides a detailed comparison between the educational network and production-grade blockchain networks.

---

[1] Introduction — Hyperledger Fabric Docs main documentation
https://hyperledger-fabric.readthedocs.io/en/latest/whatis.html

[2] [5] [7] [8] [9] [10] [11] [18] [21] [22] [24] [25] [26] [29] [30] Crypto Exchange Architecture Design: A Practical Guide for Developers - DEV Community
https://dev.to/riley_quinn_8e58a0a96d107/crypto-exchange-architecture-design-a-practical-guide-for-developers-183f

[3] [4] Blockchain KYC: The Future of Secure Customer Verification
https://authbridge.com/blog/blockchain-based-kyc/

[6] [12] [16] [17] [19] [20] [23] [43] Cryptocurrency Exchange Architecture - Easy Blueprint For Startups
https://www.coinsclone.com/cryptocurrency-exchange-architecture/

[13] [14] [15] Crypto Wallet UI Design Best Practices | IdeaSoft
https://ideasoft.io/blog/how-to-create-crypto-wallet-ui/

[27] [28] [38] [39] Building Microservices, part 8. Centralized logging with the ELK stack | Callista
https://callistaenterprise.se/blogg/teknik/2017/09/13/building-microservices-part-8-logging-with-ELK/

[31] [32] [33] [34] [35] [42] How to Design Docker Networks for Microservices
https://oneuptime.com/blog/post/2026-02-08-how-to-design-docker-networks-for-microservices/view

[36] Microservices Logging: Best Practices, Importance & Challenges
https://www.groundcover.com/microservices-observability/microservices-logging

[37] The Twelve-Factor App
https://12factor.net/logs

[40] [41] Audit Logging: What It Is & How It Works | Datadog
https://www.datadoghq.com/knowledge-center/audit-logging/